

41. First Missing Positive

Hard · Array · Hash Table

Approach — Index as a Hash Map (In-place Marking)

The key observation is that the answer must lie in the range $[1, n+1]$, where n is the size of the array. This means the array itself can be used as a hash map — index i represents whether the positive integer $i+1$ is present, encoded by flipping the sign of the value at that index.

The solution runs three clean passes over the array. First, invalid values (negatives and zeros) are replaced with $n+1$ so they don't interfere with the sign-marking scheme. Second, for each value v where $|v| \leq n$, the element at index $|v| - 1$ is negated to mark that v is present. Third, the first index with a positive value reveals the missing number.

Key Insight: After the marking pass, a positive value at index i means the number $i+1$ was never seen in the array — so $i+1$ is the answer. If all indices are marked negative, the answer is $n+1$.

Step-by-step Breakdown

Step 1 — Sanitize invalid values

Replace every non-positive number (zero or negative) with $n+1$. These values are irrelevant since the answer is in $[1, n+1]$, and $n+1$ is safely out of the marking range, so it won't corrupt the sign-encoding in the next pass.

Step 2 — Mark presence using sign flipping

For each element, take its absolute value $\text{abs}(\text{nums}[i])$ to get the original number v . If $v \leq n$, negate $\text{nums}[v-1]$ (only if it's currently positive, to avoid double-negation). After this pass, a negative value at index i means $i+1$ is present in the array.

Step 3 — Find the first unmarked index

Scan the array. The first index i where $\text{nums}[i] > 0$ means the number $i+1$ was never marked — it is the first missing positive. Return $i+1$.

Step 4 — Fallback

If every index from 0 to $n-1$ is marked (negative), then all integers 1 through n are present. The answer is $n+1$.

Solution Code

```
1  class Solution {
2  public:
3      int firstMissingPositive(vector<int>& nums) {
4          int n = nums.size();
5          for(int i = 0; i < n; i++)
6              nums[i] = nums[i] <= 0 ? n+1 : nums[i];
7
8          for(int i = 0; i < n; i++)
9          {
10             if(abs(nums[i]) <= n && nums[abs(nums[i])-1] > 0)
11                 nums[abs(nums[i])-1] *= -1;
12         }
13         for(int i = 0; i < n; i++)
14         {
15             if(nums[i] > 0) return i+1;
16         }
17         return n+1;
18     }
19 };
```

Complexity Analysis

TIME COMPLEXITY	SPACE COMPLEXITY
$O(n)$	$O(1)$
Three separate $O(n)$ passes over the array — sanitize, mark, and scan. Total remains $O(n)$.	No extra data structures are used. The array is modified in-place, giving true $O(1)$ auxiliary space.